

Graph-based Blobs Tracking

Exploiting Temporal Graphs to Estimate Trajectories

Author:

Luca Ninivaggi
s5312342

Supervisors:

Prof. Francesca Odone
Prof. Matteo Moro

Computational Vision
Academic Year 2024-2025

Abstract

Object tracking in video is a crucial step for many analysis pipelines, but it can become tricky when scenes are complex and change over time. In this paper, we introduce a graph-based framework that smooths out hiccups caused by partial occlusions and temporary mismatches that break tracking continuity. Our real-time pipeline can handle any number of blobs, and we optionally use a Kalman filter to iron out particularly noisy or erratic frames.

The highlight of our work is a novel Superblob computation on the final tracking graph: Once the video is completed, a recursive backtracking procedure merges blobs across timestamps to produce an even cleaner trajectory estimate, leveraging future frame knowledge for better accuracy. This approach results in more robust tracking, and we discuss further enhancements, to boost performance and enable real-time Superblob tracking with implemented backpropagation of knowledge.

Contents

Table of Contents	1
1 Introduction	2
1.1 Context and Motivation	2
1.2 Objectives	2
1.3 Document Organization	2
2 Background and Problem Formulation	4
2.1 Objective Definition	4
2.2 Motion Mask creation	4
2.3 Frame-wise Graph Representation	5
2.4 Bipartite Graph Between Consecutive Frames	6
2.5 Multi-layered Graph and chains extraction	7
2.6 Super-Blobs Tracking	7
3 Methods	9
3.1 Overview of the Methodology	9
3.2 Algorithmic Details	9
3.2.1 MotionDetector	9
3.2.2 CCSplitter	10
3.2.3 Matcher	10
3.2.4 Graph and Chain	11
3.2.5 KALMANObject	11
4 Experimental Results	12
4.1 Subway video	12
4.1.1 Detection	12
4.1.2 Splitting	13
4.1.3 Matching	13
4.1.4 Complete graph	14
4.1.5 Chain extraction and visualization	15
4.2 Cars in Highway	16
5 Conclusions and Future Work	17
5.1 Summary of Findings	17
5.2 Limitations	17
5.3 Recommendations for Future Research	17
A Proof that $\sim_t$ is an Equivalence Relation	19

Chapter 1

Introduction

1.1 Context and Motivation

In a previous notebook, a motion mask was computed from a given frame and a reference background image, based on two key assumptions.

- **Brightness Constancy Assumption** : The brightness of the scene does not change significantly between two consecutive frames at times t and $t + 1$.
- **Fixed Camera Assumption** : The relative position of the camera remains constant throughout the entire sequence.

Under these assumptions, the goal is to estimate the trajectory of each blob across time. To achieve this, we employ formal definitions from graph theory, connected components, and equivalence classes to construct robust object trajectories. This formulation is designed to handle challenges such as partial occlusions and temporary violations of object persistence.

1.2 Objectives

The primary objectives of this work are as follows:

1. Implement a processing pipeline that, given a binary motion mask at each time step, identifies connected components and computes their centroids.
2. Establish correspondences between connected components in consecutive frames by leveraging spatial similarity, thereby constructing a temporal graph of object detections.
3. Estimate object trajectories using a Kalman filter to handle noise, occlusions, and intermittent detection failures.
4. Provide a comprehensive visual representation of the entire pipeline, from motion detection to final trajectory estimation.

1.3 Document Organization

This document is structured as follows.

- **Chapter 2: Problem Formulation** — Introduces the theoretical background necessary to understand the proposed approach. Covers fundamental concepts such as motion detection, connected components, graph representations, and object tracking. A formal definition of the problem is also provided.

- **Chapter 3: Implementation Details** — Describes the practical realization of the proposed methodology. This includes the extraction of motion masks, the identification and analysis of connected components, the construction of the temporal graph, and the application of the Kalman filter for trajectory estimation.
- **Chapter 4: Experimental Evaluation** — Presents the experimental setup, including the video datasets used to validate the approach. It provides a detailed analysis of the results, supported by visualizations of object trajectories and discussions on accuracy and robustness.
- **Chapter 5: Conclusions and Future Work** — Summarizes the main findings, discusses the strengths and limitations of the proposed method, and suggests directions for future improvements and extensions.

Chapter 2

Background and Problem Formulation

2.1 Objective Definition

We aim to define, for each moving object $i \in \{1, \dots, N\}$, two functions:

$$F^i : \mathcal{T}^i \longrightarrow \mathbb{R}^2, \quad K^i : \mathcal{T}^i \longrightarrow \mathbb{R}^2,$$

where $\mathcal{T}^i \subseteq \{0, \dots, T\}$ are the time indices where object i is detected and tracked.

In particular the trajectories are defined such that:

- $F^i(t)$ returns the estimated 2D position of object i in the image plane at frame t ;
- $K^i(t)$ is a smoothed version of the same trajectory, e.g., obtained via filtering techniques with the Kalman filter¹ along the time dimension.

2.2 Motion Mask creation

Let $n, m \in \mathbb{N}$ and $c \in \mathbb{N}$. We define:

- $\mathcal{I}_{n,m,c} = \{ I : \{1, \dots, n\} \times \{1, \dots, m\} \rightarrow [0, 1]^c \}$
 - Set of all $n \times m$ images with c channels.
 - Each pixel value is a c -dimensional vector in $[0, 1]^c$: the intensity is normalized.
- $\mathcal{M}_{n,m} = \{ M : \{1, \dots, n\} \times \{1, \dots, m\} \rightarrow \{0, 1\} \}$
 - Set of all binary masks of size $n \times m$.
 - Each pixel is either 0 (background) or 1 (foreground).

Now, for each time frame $t = 0, 1, \dots, T$, we write:

$$I_t, B_t \in \mathcal{I}_{n,m,c}$$

- $I_t(x, y) \in [0, 1]^c$ is the observed image pixel at coordinates (x, y) .
- $B_t(x, y) \in [0, 1]^c$ is the value of the *background* at (x, y) .

¹The Kalman filter is a recursive algorithm that estimates the state of a dynamic system from a series of noisy measurements, producing smooth and robust trajectories over time.

Definition of the Motion Mask

For a *threshold* $\tau > 0$, we define the *motion mask* at each frame t as

$$M_t^\tau \in \mathcal{M}_{n,m},$$

with pixel-wise values given by

$$M_t^\tau(x, y) = \begin{cases} 1, & \text{if } \|I_t(x, y) - B_t(x, y)\| > \tau, \\ 0, & \text{otherwise.} \end{cases}$$

That is, $M_t^\tau(x, y) = 1$ only where the pixel (x, y) is different enough in the Image and the Background.

Background Update Rule

B_t is changing over time via the map

$$\varphi^\alpha : \{1, \dots, T\} \longrightarrow \mathcal{I}_{n,m,c}, \quad \alpha \in [0, 1].$$

defined as

$$\varphi^\alpha(t)(x, y) = \begin{cases} B_{t-1}(x, y), & \text{if } M_t^\tau(x, y) = 1, \\ (1 - \alpha) B_{t-1}(x, y) + \alpha I_t(x, y), & \text{if } M_t^\tau(x, y) = 0. \end{cases}$$

Consequently, the background update rule is simply

$$B_t := \varphi^\alpha(t), \quad t = 1, \dots, T.$$

2.3 Frame-wise Graph Representation

At each time t , from the binary motion mask M_t^τ we create the graph

$$G_t = (V_t, E_t),$$

where:

$$V_t = \{ (x, y) \in \{1, \dots, n\} \times \{1, \dots, m\} \mid M_t^\tau(x, y) = 1 \}$$

and

$$E_t = \{ (p, q) \mid p, q \in V_t, p \in \mathcal{N}(q) \}$$

Here $\mathcal{N}(q) \subset \{1, \dots, n\} \times \{1, \dots, m\}$ denotes the neighborhood of pixel $q = (x, y)$.

In particular, we use the *4-neighborhood*

$$\mathcal{N}_4(x, y) = \{ (x+1, y), (x-1, y), (x, y+1), (x, y-1) \},$$

So, $(p, q) \in E_t$ whenever both p and q are adjacent and have moved in the frame t .

Connected Components and Equivalence Relation

Define a binary relation $\sim_t$ on V_t by declaring $p \sim_t q$ if and only if there exists a finite path

$$\Pi = (p_0, p_1, \dots, p_k), \quad p_0 = p, \quad p_k = q,$$

such that

$$(p_i, p_{i+1}) \in E_t \quad \text{for all } i = 0, 1, \dots, k-1.$$

It can be shown that $\sim_t$ is an equivalence relation. The detailed proof is provided in Appendix A.

Blobs as Connected Components

The equivalence classes under $\sim_t$ are exactly the connected components (or *blobs*) of foreground pixels at time t .

Denote these connected components by

$$V_t / \sim_t = \{ [p]_t \mid p \in V_t \} = \{ b_t^1, b_t^2, \dots, b_t^{k_t} \} = \mathcal{B}_t$$

where each $b_t^i \subset V_t$ is a maximal set of pixels connected by edges in E_t , and $\mathcal{B}_t$ is the set of all blobs at time t .

For each blob, we may consider its induced subgraph

$$G_t[b_t^i] = (b_t^i, e_t^i), \quad e_t^i = \{ (p, q) \in E_t \mid p, q \in b_t^i \}, \quad i = 1, \dots, k_t.$$

By construction, each $G_t[b_t^i]$ is connected and contains exactly the pixels of the i -th blob at time t .

2.4 Bipartite Graph Between Consecutive Frames

Define the *bipartite graph*²

$$H_{t,t+1} = (V_{t,t+1}, E_{t,t+1}),$$

where

$$V_{t,t+1} = \mathcal{B}_t \cup \mathcal{B}_{t+1} = \{ b_t^1, b_t^2, \dots, b_t^{k_t} \} \cup \{ b_{t+1}^1, b_{t+1}^2, \dots, b_{t+1}^{k_{t+1}} \}$$

Define the *similarity-based* edge set as

$$E_{t,t+1}^{d,\delta} = \{ (a, b), (b, a) \mid a \in \mathcal{B}_t, \quad b \in \mathcal{B}_{t+1}, \quad d(a, b) > \delta \}, \quad \delta \in [0, 1].$$

where d is a *Similarity function* over the set of all *Blobs*

$$d : \mathcal{B} \times \mathcal{B} \longrightarrow [0, 1] \quad \mathcal{B} = \bigcup_{t=0}^T \mathcal{B}_t,$$

Concretely, we are defining a new graph whose nodes are the blobs at times t and $t+1$. We compute pairwise similarity scores between all blobs at adjacent time steps and retain only those connections whose similarity exceeds the threshold δ , thereby forming the edge set.

Example: Centroid-based Similarity Function

$$d_{\text{centroid}}^\sigma(a, b) = \exp \left(-\frac{\|c(a) - c(b)\|^2}{2\sigma^2} \right), \quad \sigma \in \mathbb{R}^+$$

Where c associates each Blob with its *centroid*:

$$c : \mathcal{B} \longrightarrow \mathbb{R}^2$$

since a blob is indeed a set of coordinates, i.e.

$$a = \{(x_1, y_1), \dots, (x_n, y_n)\} \subset \mathbb{R}^2$$

Then the *centroid* of a is defined as:

$$c(a) = \frac{1}{n} \sum_{i=1}^n (x_i, y_i) = \left(\frac{1}{n} \sum_{i=1}^n x_i, \frac{1}{n} \sum_{i=1}^n y_i \right) \in \mathbb{R}^2.$$

²A bipartite graph is one whose vertex set can be divided into two disjoint sets, with edges only between (not within) the sets.

2.5 Multi-layered Graph and chains extraction

We define the *multi-layered graph* H as the union of all graphs at each time step t , where

$$H = (\mathcal{B}, E), \quad \mathcal{B} = \bigcup_{t=0}^T \mathcal{B}_t, \quad E = \bigcup_{t=0}^{T-1} E_{t,t+1}.$$

In other words, H is a supergraph whose vertex set $\mathcal{B}$ contains all blobs detected across all time frames, and whose edge set E contains all edges connecting blobs between consecutive frames.

Definition of Chains

We define a *chain* γ as a sequence of blobs:

$$\gamma = (b_{t_0}, b_{t_0+1}, b_{t_0+2}, \dots, b_{t_0+k}), \quad t_0 \in \{0, \dots, T\}, \quad t_0 + k \leq T.$$

To be valid, there must exist edges that support the path; that is, we require:

$$(b_{t_0+\ell}, b_{t_0+\ell+1}) \in E \quad \text{for all } \ell = 0, \dots, k-1.$$

By the construction of the graph H , it holds that:

$$b_{t_0+\ell} \in \mathcal{B}_{t_0+\ell} \quad \text{for all } \ell = 0, \dots, k,$$

Moreover, each chain contains exactly one blob per time step:

$$\forall \gamma, \quad \forall b, b' \in \gamma, \quad b \neq b' \quad \Rightarrow \quad \nexists t \in \{0, \dots, T\} \text{ such that } b \in \mathcal{B}_t \text{ and } b' \in \mathcal{B}_t.$$

Trajectory Estimation from Chains

Therefore, for each valid chain γ^i corresponding to object $i \in \{1, \dots, N\}$, we define:

- A function

$$F^i : \mathcal{T}^i \longrightarrow \mathbb{R}^2, \quad \mathcal{T}^i = \{t_0, t_0 + 1, \dots, t_0 + k\},$$

given by

$$F^i(t) = c(b_t),$$

where $b_t \in \mathcal{B}_t$ is the unique blob in the chain γ^i at time t . This function returns the estimated position of object i in the image plane at each frame t .

- Alternatively, a trajectory

$$K^i : \mathcal{T}^i \rightarrow \mathbb{R}^2, \quad \mathcal{T}^i = \{t_0, t_0 + 1, \dots, t_0 + k\},$$

obtained by applying the *Kalman filter*.

2.6 Super-Blobs Tracking

We define the set $\mathcal{C}$ of all valid chains in H , defined as

$$\mathcal{C} = \left\{ \gamma = (b_{t_0}, b_{t_0+1}, \dots, b_{t_0+\ell}) \left| \begin{array}{l} t_0 \in \{0, \dots, T\}, \quad \ell \geq 0, \quad t_0 + \ell \leq T, \\ b_{t_0+k} \in \mathcal{B}_{t_0+k} \quad \forall k = 0, \dots, \ell, \\ (b_{t_0+k}, b_{t_0+k+1}) \in E \quad \forall k = 0, \dots, \ell - 1 \end{array} \right. \right\}.$$

Definition of Superblobs

Let $\mathcal{L} = \{\gamma^{(1)}, \gamma^{(2)}, \dots, \gamma^{(M)}\}$

be a list of M chains, where each chain is

$$\gamma^{(i)} = (b_{t_0^{(i)}}, b_{t_0^{(i)}+1}, \dots, b_{t_0^{(i)}+k^{(i)}}), \quad i = 1, \dots, M.$$

We define the function

$$\mathcal{S} : \mathcal{P}(\mathcal{C}) \rightarrow \prod_{t=0}^T \mathcal{P}(\mathcal{B}_t)$$

which maps the list of chains $\mathcal{L}$ to a sequence of *superblobs* (sets of blobs) indexed by time:

$$\mathcal{S}(\mathcal{L}) = (S_{t_{\min}}, S_{t_{\min}+1}, \dots, S_{t_{\max}}),$$

where the earliest and latest time indices in $\mathcal{L}$ as

$$t_{\min} = \min_{1 \leq i \leq M} t_0^{(i)}, \quad t_{\max} = \max_{1 \leq i \leq M} (t_0^{(i)} + k^{(i)}).$$

where each superblob at time t is

$$S_t = \bigcup_{i=1}^M \{b_t \in \gamma^{(i)} \mid t_0^{(i)} \leq t \leq t_0^{(i)} + k^{(i)}\}$$

Thus, each superblob S_t at time t aggregates all blobs that appear at that time in any chain from $\mathcal{L}$.

Analysis of Multi-Path relationships

We now define the function

$$\Phi : \mathcal{B} \rightarrow \mathcal{P}(\mathcal{C}),$$

which maps a blob $b \in \mathcal{B}$ to the set of all chains containing b :

$$\Phi(b) = \{\gamma \in \mathcal{C} \mid b \in \gamma\}.$$

This function identifies all chains passing through a given blob b , enabling multi-path analysis, caused by noise in the original images.

Trajectory of Superblobs For each superblob chain $\Gamma(b)$ corresponding to object $i \in \{1, \dots, N\}$, we define:

- A function

$$F_{agg}^i : \mathcal{T}_S^i \longrightarrow \mathbb{R}^2, \quad \mathcal{T}_S^i = \{t_{\min}, t_{\min} + 1, \dots, t_{\max}\},$$

given by

$$F_{agg}^i(t) = \frac{1}{|S_t|} \sum_{b \in S_t} c(b) \cdot |S_b|,$$

where S_t is the superblob at time t . This function returns the centroid of the superblob, representing a smoothed, aggregated position of object i in the image plane at each frame t .

- Alternatively, the trajectory

$$K_{agg}^i : \mathcal{T}_{agg}^i \longrightarrow \mathbb{R}^2, \quad \mathcal{T}_{agg}^i = \{t_{\min}, t_{\min} + 1, \dots, t_{\max}\},$$

obtained by applying a *Kalman filter* to the superblob sequence $(S_{t_{\min}}, \dots, S_{t_{\max}})$, which smooths the aggregated trajectory F_{agg}^i .

Chapter 3

Methods

3.1 Overview of the Methodology

To address the problem of motion-based object tracking, we developed a modular library organized around several key components. Each component implements a specific functionality in the tracking pipeline, from motion detection and blob extraction to graph construction, matching, and trajectory estimation.

The implementation is structured as a collection of Python classes, grouped into logical modules, each corresponding to a stage in the processing workflow:

- **Detection:** Handles the extraction of motion regions using frame differencing, and allows for a post-processing of the results.
- **Matching:** Provides predefined functions that define similarity metrics, generating edges between blobs across frames, and forming a consistent spatiotemporal graph.
- **Splitting and Aggregation:** Implements methods to split the motion masks and refine connected components, manage ambiguous regions, and eventually aggregate them into robust superblobs.
- **Graph Construction:** Explicitly builds and manipulates a multi-frame graph representation to extract chains of connected blobs, to compute trajectories.
- **Kalman Filtering:** Applies a recursive state estimator to smooth object trajectories, to handle occlusions or ambiguous detections.
- **Visualization:** Provides plotting utilities and video rendering tools for qualitative analysis and debugging.
- **Utilities:** Includes helper functions for data handling and parameter logging.

This chapter details the design and functionality of each of these modules, highlighting their role within the overall pipeline and their interdependencies.

3.2 Algorithmic Details

3.2.1 MotionDetector

The `MotionDetector` class represents the initial step of the pipeline and is responsible for computing the motion mask at a given time frame t .

It implements the previously defined functions, and additionally allows for optional post-processing operations:

- `fill_holes`: applies morphological closing to fill small gaps.
- `remove_small`: removes connected components smaller than a given size.

Implemented Detector Types The image-background difference can be computed using one of several strategies, encapsulated in the `DETECTOR` class:

- `naive_3D`: computes the pixel-wise Euclidean distance across RGB channels between the image and the background.
- `naive_1D`: converts both the image and the background to grayscale (by averaging the channels) and computes the absolute difference.

After the difference is computed, the resulting mask is thresholded using a specified value to produce the final binary motion mask.

3.2.2 CCSplitter

The `CCSplitter` class is responsible for decomposing the binary motion mask into connected components, i.e. *blobs*. Each connected component is assigned a unique integer label, producing an indexed mask and a corresponding collection of `Blob` objects.

The connected components are extracted by applying a *splitting function*, chosen from a predefined set.

Optionally, the splitter can remove inner blobs (blobs whose bounding box is completely contained within the bounding box of another blob). This is controlled by the `remove_inner` flag.

Available Splitting Methods The available splitting functions are defined in the `SPLITTER` class. Currently, the following method is implemented:

- `naive`: uses `scipy.ndimage.label` to identify connected regions in the binary motion mask. It returns:
 - a labeled mask, where each connected component is assigned a unique integer index;
 - the total number of detected components.

3.2.3 Matcher

The `Matcher` class is responsible for establishing temporal correspondences between blobs extracted at consecutive time steps t and $t + 1$.

Internally, the class constructs a similarity matrix, where each entry (i, j) measures the similarity between the i -th blob at time $t + 1$ and the j -th blob at time t . The similarity is computed using a specified metric from the `SIMILARITY` class.

Only the entries exceeding a given threshold are considered as valid matches. Optionally, the number of matches (i.e., parent edges) per blob can be limited using the `max_parents` parameter.

Similarity Metrics The similarity functions are implemented in the `SIMILARITY` class and return values in the range $[0, 1]$, where higher scores indicate more similar blobs:

- `xy`: based on the Gaussian kernel of the Euclidean distance between blob centroids.
- `rgb`: based on the Gaussian kernel of the Euclidean distance between average RGB values.
- `rgb_xy`: weighted combination of `rgb` and `xy` similarities.
- `iou`: computes the Intersection over Union (IoU) of blob masks.
- `overlap`: based on symmetric region difference.
- `xy_overlap`: combines centroid similarity and mask overlap.

3.2.4 Graph and Chain

The **Graph** class manages a temporal sequence of **Blob_collection** instances (collections of blobs detected at different timestamps) and their corresponding **EdgesCollection** that link blobs across frames.

It provides functionalities to add, validate, and retrieve blob and edge data over time.

Its main feature is extracting *chains*, i.e., ordered sequences of blobs or **SuperBlob** objects representing the evolution of tracked objects through time. Chains can be extracted in both forward and backward directions, enabling object tracking across frames.

3.2.5 KALMANObject

The **KALMANObject** class implements a basic Kalman filter for tracking and estimating the state of an object in a 2D space with position and velocity components.

It maintains matrices for the measurement model, measurement noise, process noise, and the state transition model, which can be customized via a class method.

The filter supports state prediction and update using incoming measurements, and provides a static method to merge multiple state estimates with associated covariance matrices, weighted appropriately.

This class is intended for use in tracking scenarios where sequential noisy measurements require recursive state estimation.

Chapter 4

Experimental Results

4.1 Subway video

In [This video](#) our goal is to detect and track 3 objects in time, as seen in Figure 4.1:

- The girl on the background.
- The man with the trolley.
- The other 2 men walking: a worthy observation is that it is really difficult to track the 2 of them separately, since they are of the same *blob* in a given time; we will try to deal with them using the *Kalman filtering* technique on their Superblob chain.



(a) First Frame, at $t = 0$



(b) Middle Frame, at $t = 36$



(c) Last frame, at $t = 70$

Figure 4.1: Original video sampled at different time frames

4.1.1 Detection

First, we compute the motion mask on each frame, also using some post-processing.

Parameter	Value
type	DETECTOR.naive_3D
tresh	0.2
fill_holes	3
remove_small	50
bg_rate	0.01

Table 4.1: Parameters used to compute the motion masks.

By using the parameters in Table 4.1, we obtain the results show in Figure 4.2.

The full video of motion detection along all time frames can be found at [this result](#).

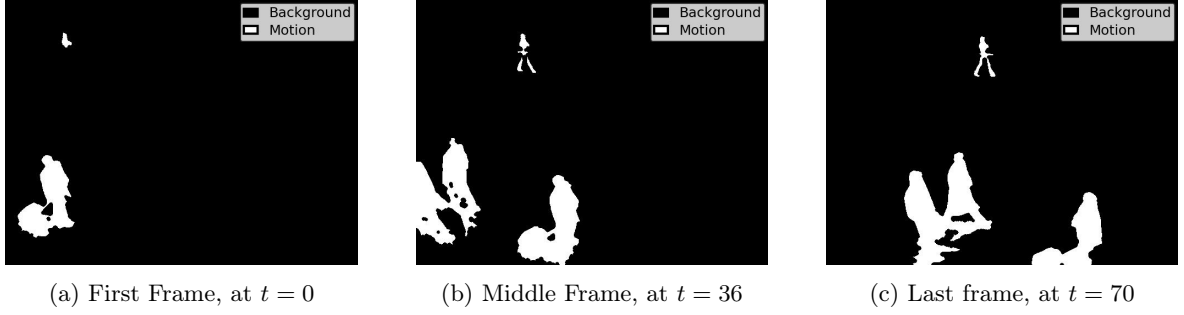


Figure 4.2: Motion mask evaluated at different time frames

4.1.2 Splitting

Once we have obtained the motion masks, we can now extract the single *Blobs*.

Parameter	Value
type	SPLITTER.naive
remove_inner	True

Table 4.2: Parameters used to compute the motion masks.

We can observe how the results perfectly match the theory: for example, as we can see in Figure 4.3, at $t = 36$ the girl is "splitted" into multiple blobs, since it is not made up of only 1 *Connected component*.

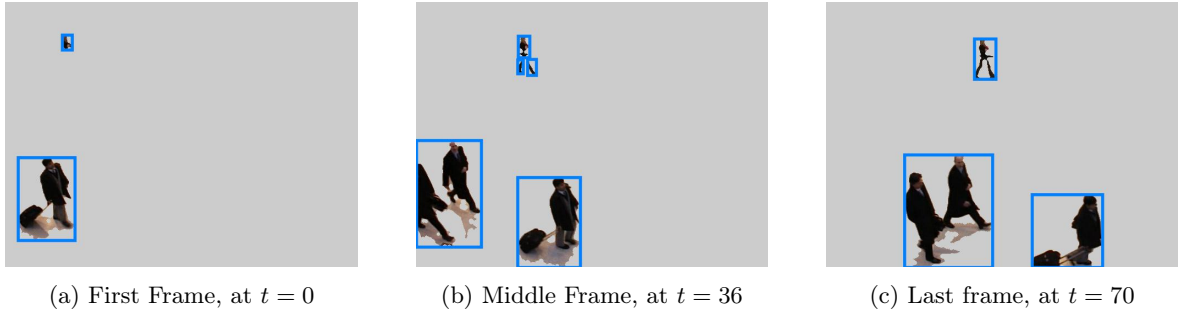


Figure 4.3: Blob extracted at different time frames

4.1.3 Matching

Now we can create the *Bipartite Graph* of subsequent frames, using the parameters shown in 4.3.

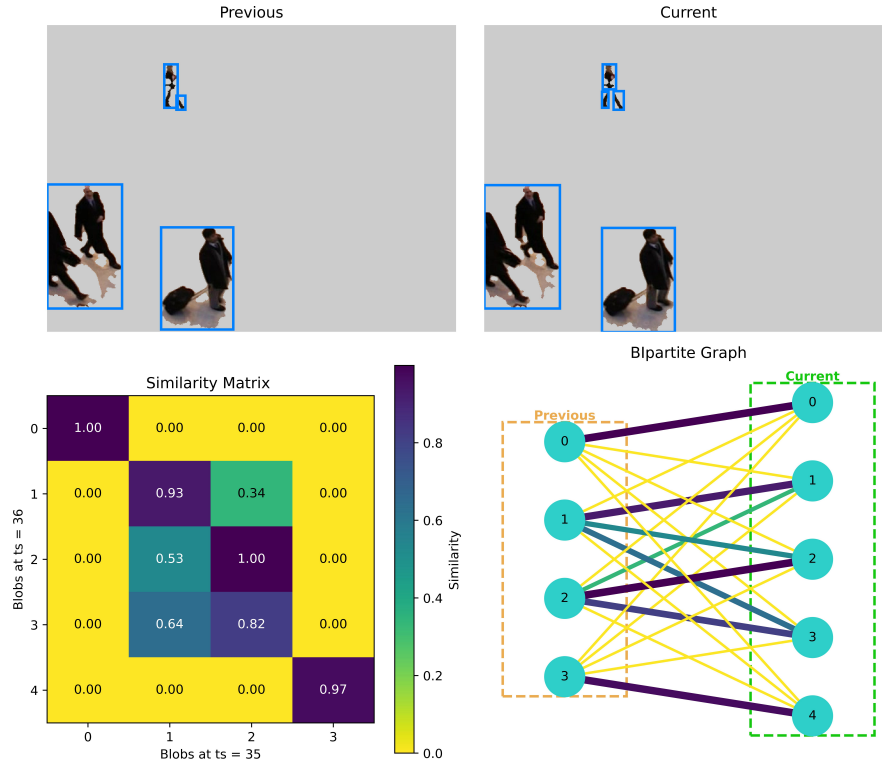
Parameter	Value
type	SIMILARITY.xy
s_xy	30
treshold	0.2

Table 4.3: Parameters used to compute the *Bipartite Graph*.

For Example, we compute $t = 36$ in Figure 4.4:

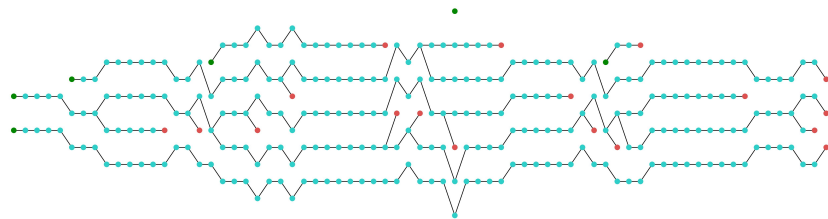
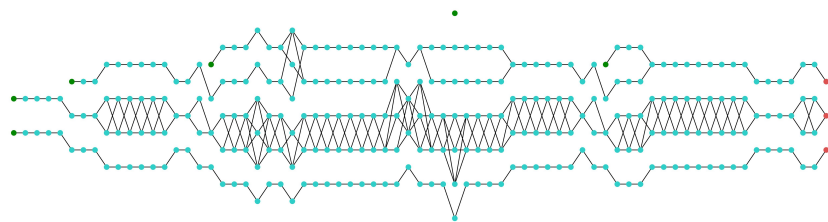
- The girl on the background is defined by the blobs b_{35}^2, b_{35}^3 in the previous time $t = 35$, and $b_{36}^1, b_{36}^2, b_{36}^3$ at current time.
- The other 2 mans walking are consistently defined by the blob with $id = 0$, i.e. b_{35}^0 and b_{36}^0 .

- The man with the trolley is described by b_{35}^3 and b_{36}^4

Figure 4.4: Matching procedure in the time frame $t = 36$

The [full video](#) shows the study along all the frames.

4.1.4 Complete graph

(a) Subgraph obtained by limiting 1 the matches ($max_parents = 1$)

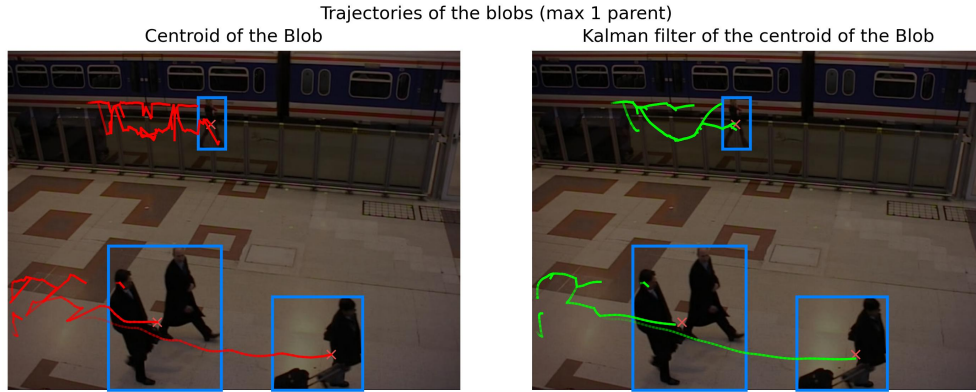
(b) Subgraph obtained by thresholding the matches

Figure 4.5: Comparison of graphs obtained by different policies

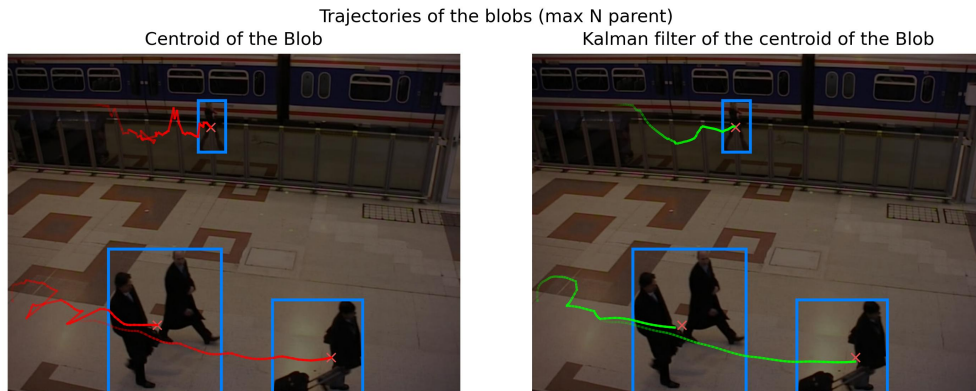
By thresholding the connections and computing all the Graphs, we can merge them to obtain a [graph](#) that visually shows the presence of *Chains*, as show in Figure 4.5.

In particular, 4.5b does not limitate the number of matches for each Blob, while in 4.5a each blob has maximum 1 matches regarding the previous frame.

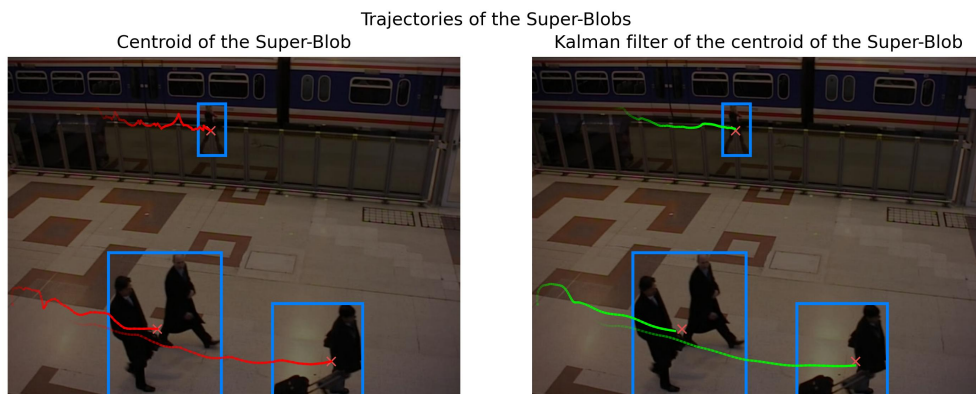
4.1.5 Chain extraction and visualization



(a) Trajectories F^i and K^i , when $max_parents = 1$



(b) Trajectories F^i and K^i



(c) Trajectories F_{agg}^i and K_{agg}^i of *Super-Blobs*

Figure 4.6: Visualization of the different paths

After the creation of the graph, we can finally show the movement of each blob, along the time axis. In Figure 4.6, all the *Trajectories* are shown.

- In Figure 4.6a, there are many trajectories, as evidenced by the large number of *ending Blobs* in Graph 4.5a; this leads to numerous inconsistencies.
- In Figure 4.6b, the number of *chains* decreases significantly, as shown in Graph 4.5b; this reduction yields better and more stable results.
- In Figure 4.6c, chain extraction is no longer an *iterative process* (unlike the two previous cases). The computation of *Superblob*, as described in Section 2.6, ensures a trajectory smoothed by the Kalman filter, producing excellent results.

The results are videos [video 1](#), [video 2](#), [video 3](#); with also the combined results for [centroids](#) and [Kalman filter](#).

4.2 Cars in Highway

Another study case is [this video](#) of cars along the highway. By using the parameters shown in Figure 4.4, we compute the trajectories, shown in 4.7.

The result is [this video](#).

Detector Parameters	
type	DETECTOR.naive_3D
tresh	0.3
fill_holes	5
remove_small	50
bg_rate	0.05
Splitter Parameters	
type	SPLITTER.naive
remove_inner	True
Similarity Parameters	
type	SIMILARITY.xy
s_xy	30
threshold	0.2
max_parents	1

Table 4.4: Configuration parameters for detector, splitter, and similarity modules.

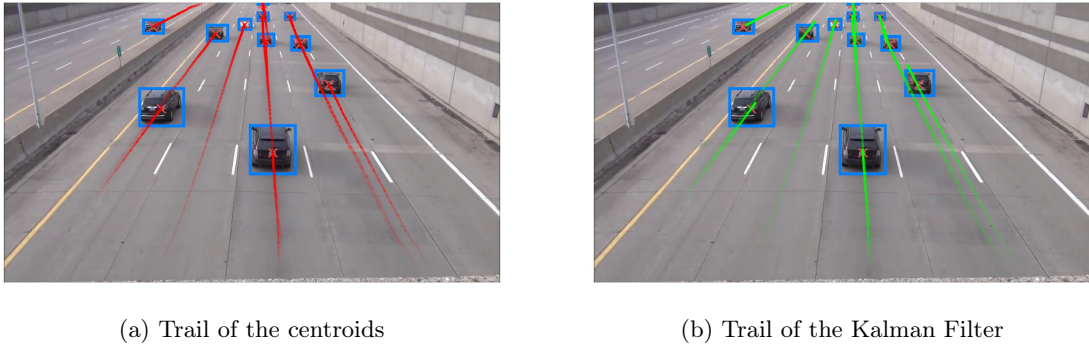


Figure 4.7: Trajectories F^i and K^i at the end of the video

Chapter 5

Conclusions and Future Work

5.1 Summary of Findings

In this work, we evaluated three different tracking models to understand their behavior under varying scene complexities: those are the model’s insights we gained about performance on videos:

- **One-to-one chaining:** In the simplest model, each blob at time $t + 1$ is matched to exactly one blob at time t . This runs in real time and works very well on sequences where blobs persist without splitting (e.g. the car-traffic video), but it breaks down under partial occlusions (as in the subway clip), leading to many lost tracks.
- **Reduced-chain approach:** By relaxing the strict one-to-one assumption, this model cuts down on fragmentation and handles minor splits better. However, when two blobs merge later on (like the two pedestrians in the subway example) the algorithm still picks the locally “best” path, resulting in chaotic, unpredictable trajectories.
- **Superblob backtracking:** Our proposed method fully addresses merge events by recursively backtracking and reprocessing all previously computed tracks whenever blobs collide. Leveraging future-frame information produces the cleanest, most coherent trajectories across all tested videos.

5.2 Limitations

Despite the clear improvements of each successive model, there are important considerations to keep in mind regarding their applicability and performance.

1. **One-to-one chaining:** Assumes no occlusions or splits; fails in complex scenes.
2. **Reduced-chain approach:** Still prone to unpredictable paths at blob-merge events.
3. **Superblob backtracking:** Requires full-video post-processing and adds computational overhead, which may not suit low-latency or streaming applications.
4. **Extensive parameter tuning:** The pipeline depends on numerous parameters, each of which can greatly influence the tracking results: the choice must be based on *prior knowledge* of the subjects, that could be not always viable. In this study, we also kept them fixed across all frames, but real-world applications might require dynamic adjustment of these parameters over time to maintain optimal performance.

5.3 Recommendations for Future Research

The object-oriented structure of the pipeline makes it naturally extensible; each module can be redefined or replaced independently, allowing for straightforward experimentation and upgrades.

Given the limited development time behind this prototype, only the core functionality and major limitations were addressed. Several directions could be explored to improve the system further:

- **Automatic parameter tuning:** A warm-up phase using the first few frames of the video could be used to estimate optimal parameters, building some form of prior knowledge about the scene before full tracking begins.
- **Dynamic parameters:** Instead of using fixed thresholds and tuning values, parameters could adapt over time based on scene activity or confidence scores, improving robustness in changing environments.
- **Hybrid execution models:** Introducing a hybrid between the second and third models could allow for more flexible chain extraction while retaining the real-time performance benefits of simpler methods. This would also help in visualizing and debugging ambiguous tracking cases.

Appendix A

Proof that $\sim_t$ is an Equivalence Relation

Proof of Reflexivity, Symmetry, and Transitivity of $\sim_t$

Reflexivity. For any $p \in V_t$, the trivial path (p) of length zero shows $p \sim_t p$.

Symmetry. Suppose $p \sim_t q$ via a path $\Pi = (p_0, p_1, \dots, p_k)$ with $p_0 = p$ and $p_k = q$. Reversing the sequence gives $(p_k, p_{k-1}, \dots, p_0)$, which is a valid path from q to p . Thus $q \sim_t p$.

Transitivity. Suppose $p \sim_t q$ via $\Pi_1 = (p_0, \dots, p_r)$ with $p_0 = p$ and $p_r = q$, and $q \sim_t r$ via $\Pi_2 = (q_0, \dots, q_s)$ with $q_0 = q$ and $q_s = r$. Concatenating Π_1 and $\Pi_2 \setminus \{q_0\}$ yields

$$(p_0, p_1, \dots, p_r = q_0, q_1, \dots, q_s),$$

which is a valid path from p to r . Hence $p \sim_t r$.